```
# OPTIMIZATION TECHNIQUE ---> it is used for minimize the error of prediction and actual value.
#(1) . gradient descent::::

# loss function = y_act-y_pred
```

```
# we know the equation of simple line::::
# y = mx+b ....(1)  where (x,y) ----> coordinates , m---> slope, b---> intercept

# y_pred = mx+b
# y_act = mx+b

# so , we will calculate loss function w.r.t means square error:::
# l= 1/n*(y_act - y_pred)**2

# now, if we have single feature then (n=1)
# l = (y_act - y_pred)**2 .....(2)

# l = (y-mx+b)**2  ....(4)
# l= (y-mx-b)**2 .....(5)

# so , we will differentiate loss w.r.t intercept(b)

# # dL/db = 2*(y-mx-b)[dy/db-dmx/db-db/db]
# dL/db = 2*(y-mx-b)[0-0-1]
# dL /db = -2(y-mx-b) .....(6)

# now , we will calculate differentiation loss w.r.t slope(m):
# dL/dm = 2*(y-mx-b)[dy/dm-dmx/dm-db/dm]
# dL/db = 2*(y-mx-b)[0-x-0]
# dL /db = -2(y-mx-b)(x) .....(7)

# now ,
# b_new = b_old - learning_rate * dl/db .....(8)
# m_new = m_old - learning_rate * dl/dm .....(9)

# if lr >>>> 0 ----> gradient exploading
# if lr <<<< 0 ----> vanishing gradient
# appropriate value of lr = 0.001

# so , our final equation of new line will be ::::
# from equ(8,9)
# y = m_new * x+ b_new ....(10)
# this is the best optimized soluction and it redice the error or minimize the loss of lines.
```

```
# (1). gradient descent ---> entire data respect ----> y = m_new*x+b_new
# (2). stocastic gradient descent ---> in this , we will select only a random row and then we calculate
# y = m_new*x+b_new

# (3) . mini batch gradient descent ---> in this , we will select our data
# if we have 100 rows then
A BATCH - 25
B BATCH - 25
```

```
C BATCH - 25
D BATCH - 25

# WE WILL SELECT a random batch using numpy random module.
# let suppose if we have b- batch , then we in this batch total 25 rowa are (x)and
# their column are m or WEIGHT , so using this data . if will reduce the loss function
# and final the best y = m_new * x _ b_new
```

```
# gradient descent optimization technique
```

```
# loss function = actual value - predicted value
# optimization = reduce the loss function
```

```
# let suppose ,if we have equation of simple line
# y= mx+b   ....(1)   (x,y)----> coordinates , m=slope(weights) ,b=intercent

# now let suppose, prediction line is :
# y_pred = mx+b ....(2)

# now , we will calculate the loss function w.r.t means squared error
# L = 1/n *(y-y_pred)**2
# n = n(features) = 1
# L = (y-y_pred)**2
# L = (y-(mx+b))**2 ......(3)

# now , we will calculate differentiation w.r.t intercept(b):
# dL/db = 2*(y-mx-b)[dy/db-dmx/db-db/db]
# dL/db = 2*(y-mx-b)[0-0-1]
# dL /db = -2(y-mx-b) .....(4)

# now , we will calculate differentiation w.r.t weights(m):
# dL/dm = 2*(y-mx-b)[dy/dm-dmx/dm-db/dm]
# dL/db = 2*(y-mx-b)[0-x-0]
# dL /db = -2(y-mx-b)(x) .....(4)
```

```
# in general lr =0.001
# b_new = b_old - learning_rate * dL/db
# m_new - m_old - learning_rate * dL/dm
```

```
# b_old = 50 , dL/db = 100
# b_new = 50-0.001*100 = 50-0.1 = 49.9
```

```
# lr >>>>0 = gradient exploading
# lr <<<<< 0 = vanishing gradient
```

```
# so , our final line of equation will be :
# y_new = m_new * x+b_new  .......(6)
```

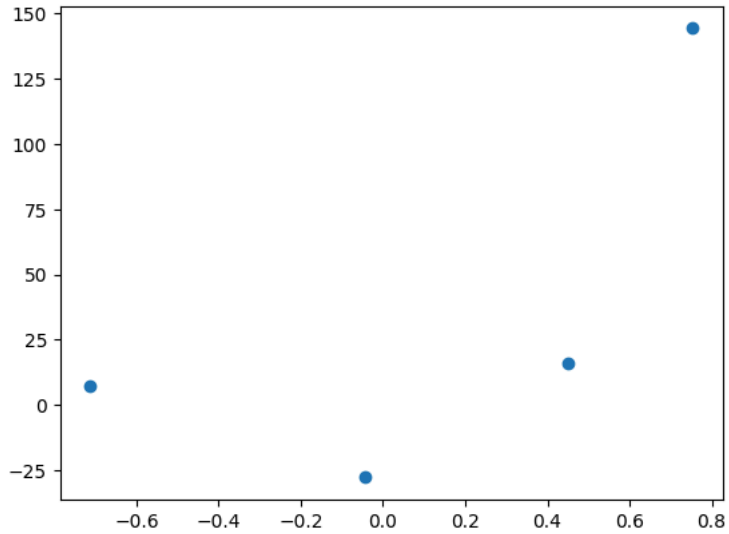
```
# equation(6) denotes as final line if equation
```

```
from sklearn.datasets import make_regression
import numpy as np
```

```
x,y=make_regression(n_samples=4,n_features=1,
                    n_informative=1,n_targets=1,
                    noise=80,random_state=13)
```

```
import matplotlib.pyplot as plt
plt.scatter(x,y)
```

```
<matplotlib.collections.PathCollection at 0x7f64110acb00>
```




```
from sklearn.linear_model import LinearRegression
```

```
reg = LinearRegression()
reg.fit(x,y)
```

```
▾ LinearRegression
LinearRegression()
```

```
LinearRegression(copy_X=True,
                 fit_intercept=True,
                 n_jobs=None)
```

```
▾ LinearRegression
LinearRegression()
```

linearregression (normalize = false)

```
reg.coef_                    # slope
```

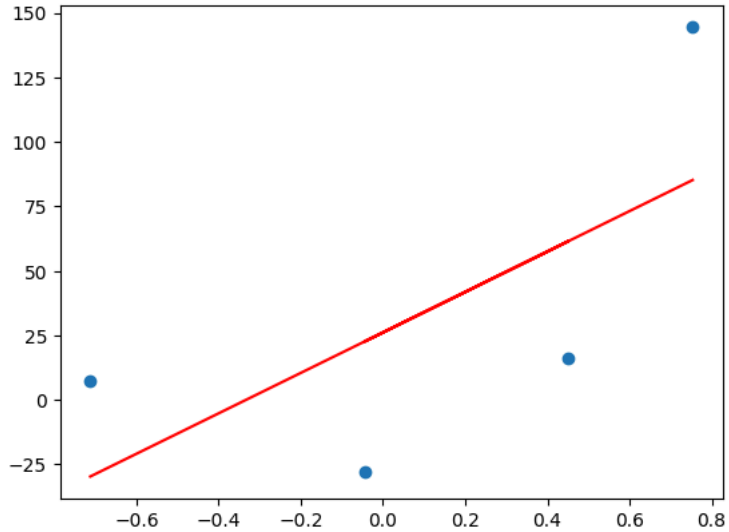
```
array([78.35063668])
```

```
reg.intercept_                       # b
```

```
np.float64(26.15963284313262)
```

```
plt.scatter(x,y)
plt.plot(x,reg.predict(x), color="Red")
```

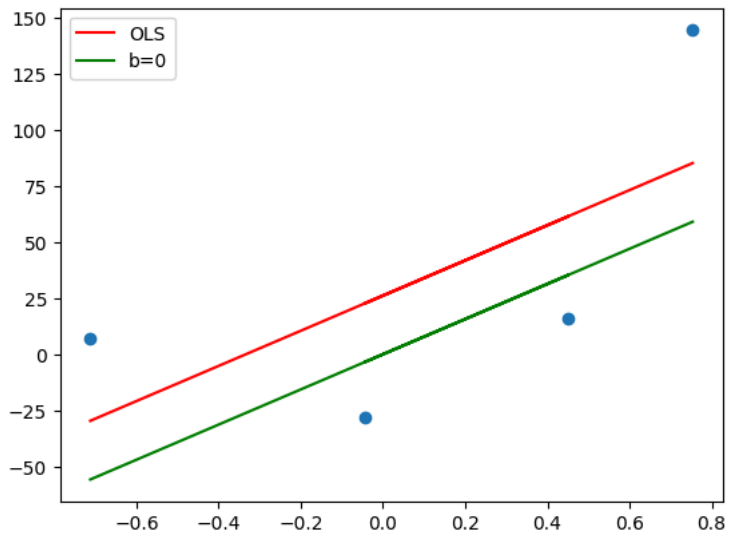
```
[<matplotlib.lines.Line2D at 0x7f640e6f6cf0>]
```




```
# lets apply gradient descent assuming slope is
# constant = 78.35
# and lets assume the starting value for
# intercept  b=0
y_pred = ((78.35*x)+0).reshape(4) # y=m*x+b
```

```
plt.scatter(x,y)
plt.plot(x,reg.predict(x), color="Red",
         labels="OLS")
plt.plot(x,y_pred, color="Green",
         labels="b=0")
```

```
plt.legend()
plt.show()
```




```
m=78.35
b=0
loss_slope = -2*np.sum(y-m*x.ravel()-b)                    # ravel high dimension into  1 dimension convert
loss_slope
```

```
np.float64(-209.27763408209216)
```

```
# lets take learning rate = 0.1
lr=0.1
step_size = loss_slope*lr
step_size
```

```
np.float64(-20.927763408209216)
```

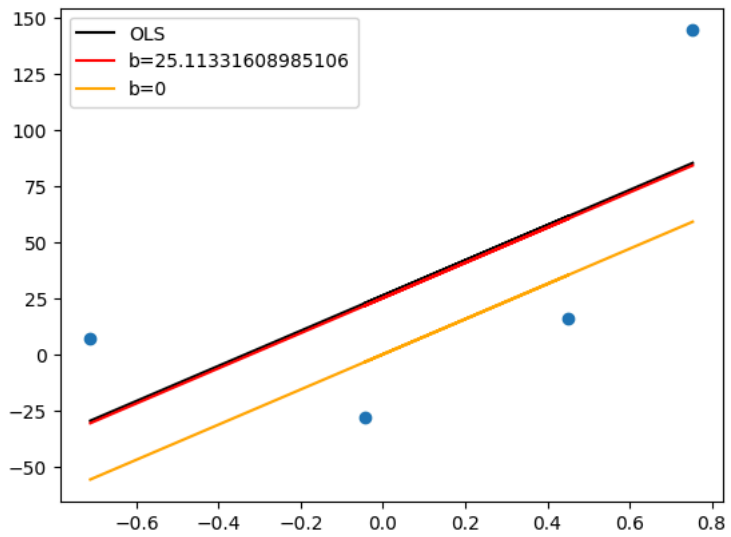
```
# calculating the new intercept
b = b-step_size                          # b_new = b_old - learning_rate * dl/db
b
```

```
np.float64(20.927763408209216)
```

```
y_pred1 = ((78.35*x)+b).reshape(4)

plt.scatter(x,y)
plt.plot(x,reg.predict(x), color="black",
         labels="OLS")
plt.plot(x,y_pred1, color="Red",
         labels="b={}".format(b))
plt.plot(x,y_pred, color="",
         labels="b=0")
```

```
plt.legend()
plt.show()
```




```
# iteration 2
loss_slope = -2*np.sum(y-m*x.ravel()-b)
loss_slope
```

```
np.float64(-41.85552681641843)
```

```
step_size = loss_slope*lr
step_size
```

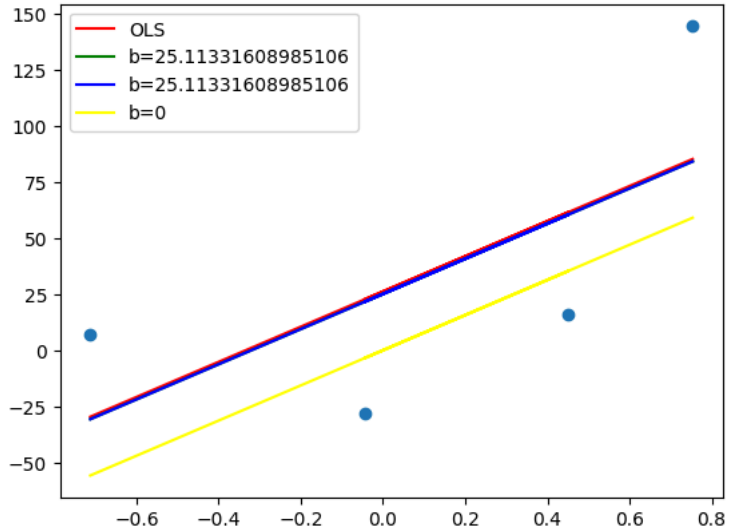
```
np.float64(-4.185552681641844)
```

```
b=b-step_size
b
```

```
np.float64(25.11331608985106)
```

```
y_pred2 = ((78.35*x)+b).reshape(4)

plt.scatter(x,y)
plt.plot(x,reg.predict(x), color="red",
         labels="OLS")
plt.plot(x,y_pred2, color="Green",
         label="b={}".format(b))
plt.plot(x,y_pred1, color="blue",
         labels="b={}".format(b))
plt.plot(x,y_pred, color="yellow",
         labels="b=0")
plt.legend()
plt.show()
```




```
# iteration 3
loss_slope = -2*np.sum(y - m*x.ravel()-2)
loss_slope
```

```
np.float64(-193.27763408209216)
```

```
step_size = loss_slope*lr
step_size
```

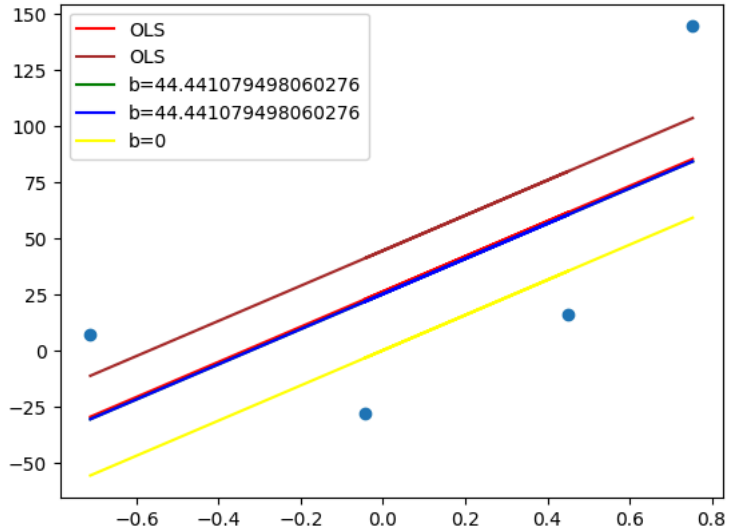
```
np.float64(-19.32776340820922)
```

```
b = b-step_size
b
```

```
np.float64(44.441079498060276)
```

```
y_pred3 =  ((78.35*x)+b).reshape(4)
```

```
plt.scatter(x,y)
plt.plot(x,reg.predict(x), color="red",
         labels="OLS")
plt.plot(x , y_pred3, color = "brown" , label="OLS")
plt.plot(x,y_pred2, color="Green",
         label="b={}".format(b))
plt.plot(x,y_pred1, color="blue",
         labels="b={}".format(b))
plt.plot(x,y_pred, color="yellow",
         labels="b=0")
plt.legend()
plt.show()
```




```
m = 78.35
b = 500
y_pred = ((78.35*x)+b).reshape(4)
```

```
m = 75.38
b = 500
loss_slope = -2*np.sum(y - m*x.ravel()-b)
loss_slope
```

```
np.float64(3788.0571771544273)
```

```
lr = 0.1
step_size = loss_slope*lr
step_size
```

```
np.float64(378.80571771544277)
```

```
b = b-step_size
b
```

```
np.float64(121.19428228455723)
```

```
loss_slope = -2*np.sum(y-m*x.ravel()-b)                    # ravel high dimension into  1 dimension convert
loss_slope
```

```
np.float64(757.6114354308854)
```

```
step_size = loss_slope*lr
step_size
```

```
np.float64(75.76114354308855)
```

```
b = b-step_size
b
```

```
np.float64(45.43313874146868)
```

```
loss_slope = -2*np.sum(y-m*x.ravel()-b)                    # ravel high dimension into  1 dimension convert
loss_slope
```

```
np.float64(151.5222870861772)
```

```
step_size = loss_slope*lr
step_size
```

```
np.float64(15.15222870861772)
```

```
b = b-step_size
b
```

```
np.float64(30.28091003285096)
```

```
loss_slope = -2*np.sum(y-m*x.ravel()-b)                    # ravel high dimension into  1 dimension convert
loss_slope
```

```
np.float64(30.30445741723544)
```

```
step_size = loss_slope*lr
step_size
```

```
np.float64(3.030445741723544)
```

```
b = b-step_size
b
```

```
np.float64(27.250464291127418)
```

```
loss_slope = -2*np.sum(y-m*x.ravel()-b)                    # ravel high dimension into  1 dimension convert
loss_slope
```

```
np.float64(6.060891483447051)
```

```
step_size = loss_slope*lr
step_size
```

```
np.float64(0.6060891483447052)
```

```
b = b-step_size
b
```

```
np.float64(26.644375142782714)
```